MagArrow Heading Error Calibration / Compensation Program

사용자 매뉴얼

버전 1.0 | 2026-08-04

설치 위치: `C:\Mag_Heading`

1. 개요

이 프로그램은 Geometrics MagArrow 세슘 자력계 시스템의 heading error(방위 오차)를 보정하기 위한 독립 실행 도구다. Geometrics가 제공하는 LabVIEW 기반 "Survey Data Processing" 프로그램은 5개 모듈(1) Survey Data Preparation, 2) Survey Data Noise Reduction, 3) Heading Error Calibration File Generation, 4) Heading Error Compensation, 5) Survey Data Combination)로 구성되는데, 이 중 3) 캘리브레이션 파일 생성 단계는 GUI 커서로 구간을 수작업 선택해야 하고 품질 판단 기준(자기평탄도, 헤딩 커버리지)이 서로 상충해서 좋은 결과를 내기가 쉽지 않다.

이 프로그램은 벤더 프로그램의 3), 4) 단계에 해당하는 부분만 더 정량적인 방식으로 재구현한다. 1) 베이스국 보정, 2) 노이즈 저감/Line Leveling, 5) 파일 결합은 계속 벤더 프로그램을 그대로 사용하며, 이 프로그램은 그 사이(3, 4단계)에만 끼워 넣는다.

1.1 왜 만들었나

벤더 매뉴얼과 이론적 근거가 되는 논문(Zhang et al., 2022, The Leading Edge, "Motion noise reduction in drone magnetometry using a dead-zone-free total-field magnetometer system")을 검토한 결과, 다음 두 가지를 개선할 여지를 발견했다:

- 캘리브레이션 구간의 품질(자기평탄도, 헤딩 커버리지)을 **수치로** 평가할 수 있게 만들면 GUI 커서로 눈대중 선택하는 것보다 훨씬 재현 가능한 캘리브레이션 파일을 만들 수 있다.
- 논문은 벤더 프로그램의 기본값인 nearest-neighbor 보간 대신 biharmonic spline 보간을 쓰면 성능이 개선된다고 보고한다 — 이 프로그램은 두 방법을 모두 제공해 실측 비교가 가능하다.
- 좋은 캘리브레이션 구간을 수작업으로 찾는 대신, 전체 취득 자료를 슬라이딩 윈도우로 스캔해서 후보 구간을 자동으로 랭킹해주는 기능(`scan`)을 추가했다.

1.2 전체 데이터 흐름

단계	내용
1	MagArrow 원본 자료 → Survey Manager로 .csv 변환
2	벤더 프로그램 Module 1 (Survey Data Preparation): 구간 자르기 + 베이스국 보정 → `<survey>-pre.csv` (Mag, CompassX/Y/Z, GPS/IMU/NMEA 등 62열)
3	(선택) `mag_heading scan` → 후보 캘리브레이션 구간 랭킹
4	`mag_heading calibrate` → `project.cal.npz` (프로젝트당 1개, 재사용)
5	`mag_heading compensate` (4의 .npz 사용) → `<survey>-pre-comp.csv` (MagComp 컬럼 추가)
6	벤더 프로그램 Module 2 (노이즈 저감/Line Leveling), Module 5 (파일 결합)

핵심은 **하나의 프로젝트(측선 조사)마다 캘리브레이션 파일을 한 번만 만들고**, 그 파일로 해당 프로젝트에서 취득한 모든 `-pre.csv`를 compensation한 뒤 벤더 프로그램의 나머지 단계로 넘기는 것이다.

2. 설치

Python 3.10 이상이 필요하다. `C:\Mag_Heading`에서:

```
cd C:\Mag_Heading
pip install -r requirements.txt
pip install -e .
```

설치되는 주요 패키지: numpy, scipy, pandas, matplotlib. GUI는 Python 표준 라이브러리인 Tkinter를 사용하므로 별도 설치가 필요 없다.

주의: `requirements.txt`는 `numpy<2.3`으로 버전을 고정해 두었다. 이는 scipy 1.13.1과의 바이너리 호환성 때문이며 (scipy가 numpy<2.3을 요구), 이 핀을 임의로 제거하면 계산 결과가 조용히 깨질 수 있으니 건드리지 않는다.

설치가 끝나면 `python -m mag_heading --help`로 확인한다:

```
usage: mag_heading [-h]
                   {calibrate,check,compensate,scan,plot2d,plot-heading,
                    plot-calibration,gui}
                   ...
```

3. 데이터 준비

이 프로그램은 벤더 프로그램 Module 1(Survey Data Preparation)이 만든 `-pre.csv` 파일을 입력으로 받는다. 이 단계에서 이미 구간 자르기(이륙 전/착륙 후 제거)와 베이스국 보정(diurnal variation 제거)이 끝나 있어야 한다.

3.1 필수 컬럼

아래 4개 컬럼이 반드시 있어야 한다 (나머지 컬럼은 그대로 보존되어 출력 파일에도 남는다):

옵션	설명
Mag	정밀 total-field 자력계 판독값 (nT)
CompassX / CompassY / CompassZ	방위 산출용 3축 보조 나침반 raw 값. 절대 스케일은 중요하지 않고 방향만 사용한다.

2D 지도 플롯(`plot2d`)을 쓰려면 `Latitude/Longitude` 컬럼도 필요하다.

3.2 헤딩(θ, φ) 정의

CompassX/Y/Z가 이루는 벡터 B에 대해, polar angle θ는 B와 z축 사이각, azimuth φ는 B의 x-y 평면 투영과 x축 사이각으로 정의한다 (벤더 매뉴얼 부록과 동일한 정의). 이 두 각도의 조합이 heading error ΔB(θ, φ)를 결정한다 — 자세한 이론은 8장 부록 참고.

4. 명령어 레퍼런스

모든 명령은 `python -m mag_heading <subcommand> [options...]` 형태다.

4.1 scan — 캘리브레이션 후보 구간 자동 탐색

고정 크기 윈도우를 전체 `-pre.csv` 위에서 슬라이딩하며, 각 윈도우를 캘리브레이션 구간으로 썼을 때의 자기평탄도(self-compensation p2p), 전체 survey 대비 헤딩 커버리지, 그리고 **survey의 주 헤딩(드론의 주 비행 방향) 커버 여부**를 계산해 순위를 매긴다. 순위는 주 헤딩 커버 여부를 최우선으로, 그다음 자기평탄도로 정렬한다 — 자기평탄도가 아무리 좋아도 주 비행 방향을 놓치면 그 오차가 survey 전체에 반복되므로 더 치명적이기 때문이다. **좋은 구간을 눈으로 찾기 어려울 때 가장 먼저 실행할 명령.**

```
python -m mag_heading scan --pre <survey>-pre.csv --window-size 500 \
    --max-variation 2.0
```

옵션	설명	
--pre	입력 -pre.csv 경로 (필수)	
--window-size	후보 윈도우 크기, 샘플 수 (필수)	
--step	윈도우 간 이동 간격	window-size // 4
--method	nearest 또는 spline	nearest
--coverage-threshold-deg	헤딩 커버리지 판정 각도 임계값(도)	10.0
--grid-resolution-deg	내부 각도 그리드 해상도(도)	5.0
--trim-outliers / --outlier-mad-threshold	4.2와 동일	꺼짐 / 3.0
--max-variation	자기평탄도 pass/fail 임계값(nT)	None (판정 생략)
--top-n	출력할 후보 개수	10

출력 표의 `dom_cov` 컬럼이 그 윈도우가 survey의 주 헤딩을 커버하는지 여부다. `--trim-outliers`를 주면 `trimmed` 컬럼에 각 후보에서 제거된 이상치 개수가 함께 표시된다.

4.2 calibrate — 캘리브레이션 모델 생성

선택한 구간(인덱스 범위 또는 HH:MM:SS 시각)으로 캘리브레이션 모델을 만들어 `.npz` 파일로 저장한다.

```
python -m mag_heading calibrate --pre <survey>-pre.csv \
    --start 2800 --end 3299 --method spline --trim-outliers \
    --out project.cal.npz --max-variation 2.0
```

옵션	설명	
--pre	입력 -pre.csv 경로 (필수)	
--start / --end	구간의 시작/끝. 정수 인덱스 또는 HH:MM:SS 시각 (필수)	
--method	nearest 또는 spline	nearest
--trim-outliers	구간 내 각도 그리드 칸(bin)마다 heading과 무관해 보이는 튀는 값을 MAD(median absolute deviation) 기준으로 제거. 칸을 통째로 비우지는 않으므로 특정 헤딩의 유일한 관측치는 튀어 보여도 보존된다	꺼짐
--outlier-mad-threshold	--trim-outliers의 robust z-score 임계값	3.0
--out	출력 캘리브레이션 모델 경로 (.npz) (필수)	

옵션	설명	
--export-vendor-cal	벤더 -cal.csv 형식(Mag,CompassX,CompassY,CompassZ)으로도 내보내기	-
--coverage-threshold-deg / --grid-resolution-deg / --max-variation	4.1과 동일	10.0 / 5.0 / None

캘리브레이션 자료는 많을수록 좋은가? 아니다. 벤더 매뉴얼과 이론적 근거 논문(Zhang et al.) 모두 자기평탄도와 헤딩 커버리지가 서로 상충한다고 명시한다. 중요한 건 자료 개수가 아니라 ① 헤딩 다양성(특히 survey의 주 비행 방향이 반드시 담길 것)과 ② 그 자기장 변화가 순전히 heading 때문이어야 한다는 것이다. 자세한 근거는 8.5절 참고.

4.3 check — 캘리브레이션 품질 확인

저장된 캘리브레이션 모델의 품질 지표를 출력한다. `--pre`를 함께 주면 그 survey 파일의 헤딩 분포 대비 커버리지도 계산한다 (지표 해석은 6장 참고).

```
python -m mag_heading check --cal project.cal.npz \
    --pre <survey>-pre.csv --max-variation 2.0
```

4.4 compensate — 자료 보정 적용

캘리브레이션 모델을 survey 파일 전체에 적용해 `MagComp` 컬럼을 추가한 `-comp.csv`를 만든다. 원본 스키마(모든 열)를 그대로 보존하므로 벤더 프로그램 Module 2에 바로 넣을 수 있다.

```
python -m mag_heading compensate --pre <survey>-pre.csv \
    --cal project.cal.npz
```

옵션	설명	
--pre / --cal	입력 survey 파일 / 캘리브레이션 모델 (필수)	
--out	출력 경로	-comp.csv
--overwrite	MagComp를 추가하는 대신 Mag 컬럼을 직접 덮어씀 (원본 raw 값 소실)	끔

4.5 시각화: plot-heading / plot-calibration / plot2d / plot-timeseries

구간 선택 전 전체 헤딩 흐름을 보려면 `plot-heading`, 캘리브레이션 자기평탄도(raw vs self-compensated, 캘리브레이션 구간 자체만)를 보려면 `plot-calibration`, compensation 전후 2D 지도 비교(Latitude/Longitude 필요)는 `plot2d`를 쓴다. `plot-timeseries`는 **compensation을 적용한 survey 파일 전체**에서 raw와 compensated를 시간축에 겹쳐 그린다(벤더 프로그램 Module 4 "MagArrow Data Plot" 스타일) — 캘리브레이션 구간만 보여주는 plot-calibration과 달리, 실제 survey 전체에 compensation이 얼마나 효과가 있었는지 확인하는 용도다.

```
python -m mag_heading plot-heading --pre <survey>-pre.csv --out heading.png

python -m mag_heading plot-calibration --cal project.cal.npz --out cal_quality.png

python -m mag_heading plot2d --pre <survey>-pre-comp.csv --out map.png

python -m mag_heading plot-timeseries --pre <survey>-pre-comp.csv --out timeseries.png
```

참고: heading effect는 보통 수 nT 이내인데 실제 지자기 신호는 수십~수백 nT 범위라, `plot-timeseries`를 전체 survey 스케일로 보면 raw/compensated가 거의 겹쳐 보일 수 있다 — 확대(줌인)해서 봐야 차이가 드러난다.

4.6 batch-compensate — 여러 파일 일괄 처리 + 개선 확인

캘리브레이션 파일을 만든 뒤, 현장에서 취득한 모든 파일에 한꺼번에 적용해서 파일별로 실제 개선 정도를 확인한다. 헤딩 오차 자체보다 **스윙(swing)에 의한 값의 기복을 줄이는 것**이 이 도구의 핵심 목적이므로, 원시 표준편차가 아니라 sample-to-sample 1차 차분의 표준편차(`motion_noise_std_nT`)로 개선율을 측정한다 — 실제 이상체 신호(느리게 변하는 성분)에는 오염되지 않는 지표다.

```
python -m mag_heading batch-compensate \
    --pre-glob "D:/HaeNam_Mag/20260715/*-pre.csv" \
    --cal project.cal.npz --out-dir D:/HaeNam_Mag/20260715/compensated
```

옵션	설명	기본값
--pre	-pre.csv 경로 여러 개(공백으로 구분)	-
--pre-glob	-pre.csv glob 패턴 (예: 'D:/survey/*-pre.csv')	-
--cal	캘리브레이션 모델 경로 (필수)	
--out-dir	출력 -comp.csv를 모아둘 폴더	각 입력 파일 옆
--overwrite	4.4와 동일	꺼짐

파일별 개선율(%)이 표로 출력되고, 5% 미만인 파일은 별도로 표시된다. 여러 파일에 걸쳐 개선율이 고르게 좋으면 그 캘리브레이션 파일을 채택하고, 특정 파일에서만 개선이 약하면 그 구간의 헤딩이 캘리브레이션에 충분히 담기지 않았을 가능성이 높다 — `scan`으로 다시 구간을 찾아본다.

4.7 gui — 경량 데스크톱 GUI

터미널 없이 쓰려면 `Run MagHeading GUI.bat`을 더블클릭한다. 명령줄로 직접 실행하려면:

```
python -m mag_heading gui --pre <survey>-pre.csv
```

자세한 사용법은 7장 참고.

5. 실전 워크플로 예제

HaeNam 샘플 데이터(`SRVY0-ACQU429-20Hz-LPF-pre.csv`, 4598 샘플, 20 Hz)로 처음부터 끝까지 실행한 실제 예시다.

5.1 헤딩 시계열로 전체 흐름 확인

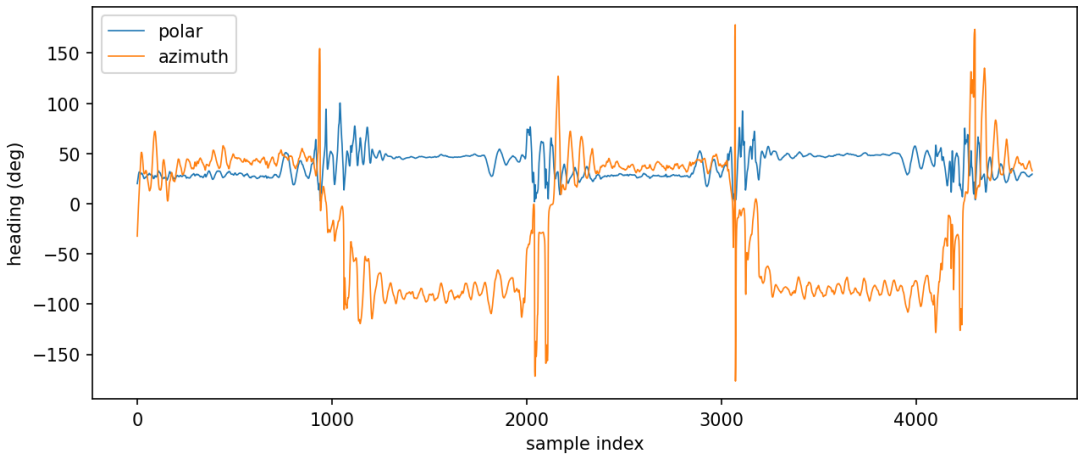


그림 1. 전체 취득 구간의 polar/azimuth 헤딩. index ~950/2050/3050/4200 부근마다 azimuth가 크게 요동치는 반복 패턴이 보인다 — 반복된 선회/캘리브레이션 구간의 흔적으로 추정된다.

5.2 scan으로 최적 구간 탐색

```
$ python -m mag_heading scan --pre SRVY0-ACQU429-20Hz-LPF-pre.csv \
    --window-size 500 --max-variation 2.0

   start      end     p2p_nT  uncovered  grid_bins   pass
    2875     3374      2.352    0.06786        124  False
    1750     2249      2.649    0.05307        134  False
     750     1249      2.892    0.01979        170  False
    2750     3249      2.986     0.1601        116  False
    3875     4374      3.108    0.04502        160  False
```

가장 위 후보(index 2800~3299 부근)를 채택한다. 자기평탄도 순으로 정렬되며, pass 컬럼은 `--max-variation` 임계값 통과 여부다 — 이 예시에서는 전부 False지만 이는 임계값 2.0 nT가 이 특정 데이터셋 기준으로는 다소 엄격했기 때문이고(6장 참고), 상대적 순위 자체가 중요하다.

5.3 calibrate + check

```
$ python -m mag_heading calibrate --pre SRVY0-ACQU429-20Hz-LPF-pre.csv \
    --start 2800 --end 3299 --method spline \
    --out project.cal.npz --max-variation 2.0

calibration model saved to project.cal.npz (500 samples, method=spline)
  self_compensation_p2p_nT: 1.774
  self_compensation_pass: True

$ python -m mag_heading check --cal project.cal.npz \
    --pre SRVY0-ACQU429-20Hz-LPF-pre.csv --max-variation 2.0

  survey_uncovered_fraction: 0.07112
  survey_chord_distance_median: 0.03664
```

같은 구간을 `nearest`로 만들면 p2p 2.38 nT, `spline`은 1.77 nT — spline이 더 우수했다 (Zhang et al. 논문의 주장과 일치).

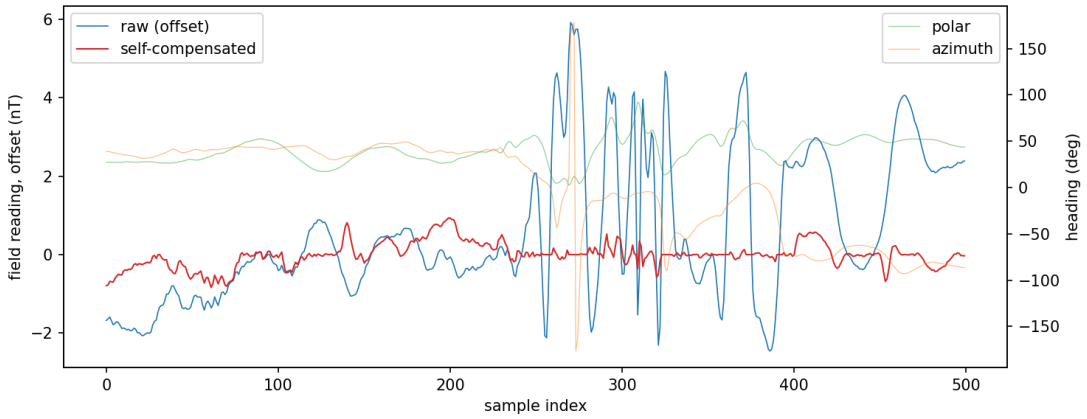


그림 2. 캘리브레이션 구간의 raw(파랑) vs self-compensated(빨강) 판독값, 헤딩(녹색/주황, 오른쪽 축) 중첩. 빨간 선이 대체로 평평할수록 그 구간의 자기장 변화가 heading effect만으로 잘 설명된다는 뜻이다.

5.4 compensate + plot2d

```
$ python -m mag_heading compensate --pre SRVY0-ACQU429-20Hz-LPF-pre.csv \
    --cal project.cal.npz

compensated survey written to SRVY0-ACQU429-20Hz-LPF-pre-comp.csv (4598 samples)

$ python -m mag_heading plot2d --pre SRVY0-ACQU429-20Hz-LPF-pre-comp.csv \
    --out map.png
```

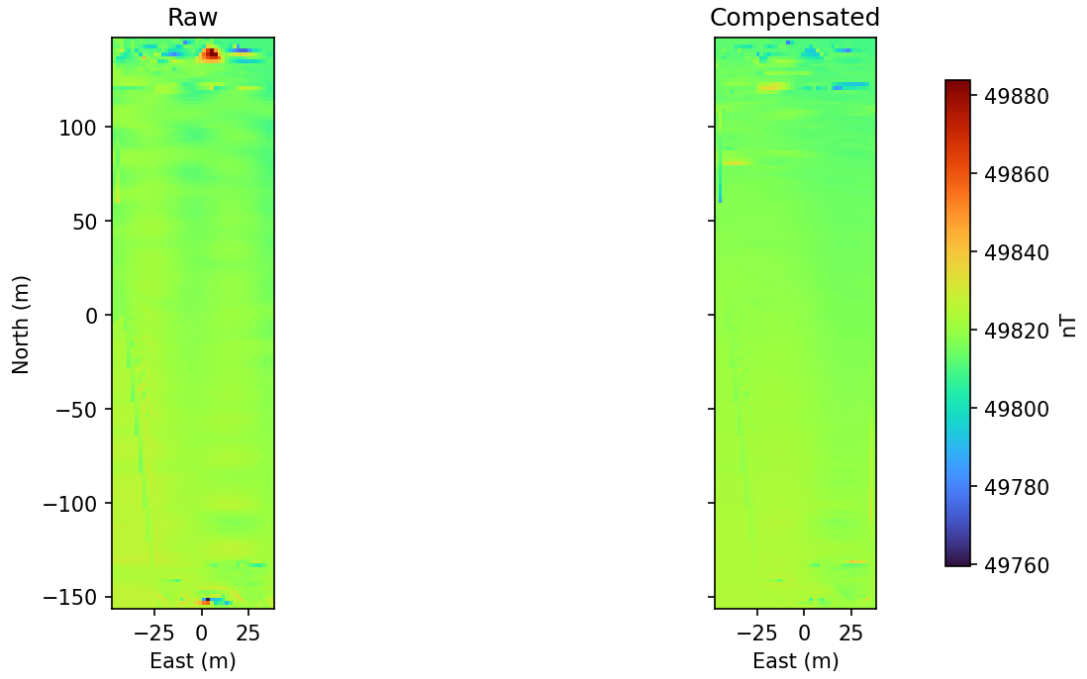


그림 3. compensation 전(좌)/후(우) 2D 지도. 좌측 상단의 heading-artifact성 hotspot이 우측에서 뚜렷이 줄어든 것을 볼 수 있다. 전체 sample-to-sample 노이즈는 약 23% 감소했다.

이 `-comp.csv`를 벤더 프로그램 Module 2(Survey Data Noise Reduction/Line Leveling)에 입력해 나머지 처리를 이어간다.

6. 품질 지표 해석 가이드

`check`와 `calibrate`가 출력하는 지표들의 의미다.

옵션	설명
self_compensation_p2p_nT	캘리브레이션 구간 자체에 모델을 적용했을 때 잔차의 peak-to-peak. 작을수록 그 구간의 자기장 변화가 heading effect만으로 잘 설명된다는 뜻. 값이 크면 공간적 자기장 구배, 드론 간섭, 불완전한 베이스국 보정 등 heading과 무관한 오염이 섞여 있다는 신호.
self_compensation_pass	`--max-variation` 임계값 이하인지 여부. 임계값은 survey 목표(찾으려는 이상체의 신호 크기)에 맞춰 사용자가 정한다.
n_grid_bins / median_samples_per_bin	캘리브레이션 데이터를 각도 그리드로 나눴을 때 채워진 칸 수와 칸당 중앙값 샘플 수. bin 수가 너무 적으면 헤딩 해상도가 부족한 것이고, 칸당 샘플이 1개뿐이면(아래 항목) 그 방향에서는 heading-only 가정을 검증할 반복 관측이 없다는 뜻.
bins_with_single_sample_fraction	샘플이 1개뿐인 grid bin의 비율. 이 비율이 높으면 self_compensation_p2p 지표의 신뢰도가 낮아진다(비교할 반복 관측이 부족하므로) — 구간을 늘리거나 더 느리게/여러 번 선회하는 캘리브레이션 비행이 필요할 수 있다.
survey_uncovered_fraction	survey 전체 샘플 중, 캘리브레이션 데이터가 커버하는 헤딩과 `--coverage-threshold-deg` 이상 떨어진 샘플의 비율. 높으면 survey에서 실제 겪는 자세 변화를 캘리브레이션 비행이 충분히 포함하지 못했다는 뜻이다.
survey_chord_distance_median / p95 / max	survey 샘플들이 가장 가까운 캘리브레이션 grid bin까지 떨어진 각도 거리(3D 단위구 chord 거리)의 분포. p95/max가 크면 일부 극단적인 헤딩에서 외삽(nearest-neighbor로 fallback)이 일어나고 있다는 신호.
survey_dominant_heading_deg / _fraction	survey에서 가장 많이 관측된 (polar, azimuth) — 대부분의 survey는 선회보다 직선 비행 시간이 훨씬 길기 때문에, 이는 곧 드론의 주 비행 방향에 대한 근사치다. _fraction은 그 방향이 survey 샘플의 몇 %를 차지하는지.
survey_dominant_heading_covered	위 주 헤딩이 캘리브레이션에 실제로 커버되는지 여부. **이게 False면 다른 지표가 다 좋아도 재검토가 필요하다** — 그 오차가 survey 거의 전체에서 반복되기 때문이다.
n_samples_before_trim / outlier_trim_stats	`--trim-outliers` 사용 시, 트리밍 전 샘플 수와 제거된 샘플 수·비율(모델 메타데이터에 저장됨).

참고: self_compensation_p2p는 잔차 중 딱 하나의 최댓값에 좌우되는 극단값 통계라 트리밍 후에 오히려 미세하게 나빠질 수도 있다(어느 bin의 평균이 재계산되면서 다른 점의 잔차가 커지는 경우) — 이런 경우 self_compensation_std_nT(표준편차, 극단값에 덜 민감)가 실제로 개선됐는지 함께 확인한다. HaeNam 샘플 데이터에서는 트리밍 후 std가 nearest 0.3699→0.3648, spline 0.3147→0.3105로 두 방법 모두 개선됐다.

6.1 좋은 캘리브레이션 구간을 고르는 팁

- `scan`을 먼저 돌려서 주 헤딩을 커버하면서 자기평탄도가 가장 좋은 후보들을 얻는다 (dom_cov 컬럼 확인).
- 상위 후보들의 `survey_uncovered_fraction`도 함께 확인한다 — 자기평탄도가 가장 좋아도 헤딩 커버리지가 너무 좁으면(예: 한쪽 방향으로만 계속 도는 구간) survey 전체를 잘 보정하지 못할 수 있다.
- 벤더 매뉴얼이 말하듯 두 기준(평탄도·커버리지)은 서로 상충하는 경우가 많다 — 구간을 넓히면 커버리지는 늘지만 평탄도가 나빠지기 쉽다. 여러 후보를 `calibrate` + `check`로 실제 비교해보는 것이 가장 확실하다.
- 같은 방향으로 여러 번 선회하는 구간은 `bins_with_single_sample_fraction`을 낮춰서 self-compensation 지표의 신뢰도를 높인다.
- `--trim-outliers`는 '자료를 더 늘리는' 대신 '이미 있는 자료에서 heading과 무관한 잡음만 골라내는' 접근이다 — 구간을 넓혀서 커버리지를 늘리되, 그 안의 튀는 값은 트리밍으로 걸러내는 조합이 두 기준을 동시에 만족시키는 실전 전략이다.

- 캘리브레이션을 만든 뒤에는 반드시 `batch-compensate`로 실제 현장 파일들에 적용해 개선율을 확인한다(4.6절) — self-compensation 지표만으로는 알 수 없는, 그 캘리브레이션이 실제 survey 전반에 얼마나 잘 일반화되는지를 보여준다.

7. GUI 사용법

`C:\Mag_Heading` 폴더의 `Run MagHeading GUI.bat`을 더블클릭하면 터미널 명령 없이 바로 실행된다. 바탕화면에 바로가기를 만들어 두면 이후로는 그 아이콘만 더블클릭하면 된다.

명령줄에서 직접 실행하려면 `python -m mag_heading gui --pre <survey>-pre.csv`. Tkinter 기반이라 별도 설치가 필요 없다(표준 라이브러리).

7.1 화면 구성

- **상단**: -pre.csv 파일 로드 버튼
- **왼쪽 탭 4개**: "Segment Selection"(Mag 시계열 + **GPS 비행 경로 플롯이 나란히 표시되고, 선택 구간이 경로 위에 빨간 점으로 하이라이트** — 벤더 프로그램의 커서 조작과 동일한 방식에 측선상 위치 확인 기능을 더한 것; GPS 좌표가 없는 파일은 이 패널이 생략됨). 오른쪽 패널의 "Add segment" 버튼으로 드래그 선택을 계속 추가할 수 있고, 추가된 구간은 시계열 위에 초록 띠로, 경로 위에는 초록 점으로 표시된다(아직 추가 안 한 현재 드래그는 빨간 점) — 8.6절 다구간 캘리브레이션 기능의 GUI 버전. "Calibration Quality"(그림 2와 동일한 자기평탄도 플롯 — 캘리브레이션 구간 자체만), "2D Comparison"(compensation 적용 후 **raw vs compensated 시간영역 그래프가 위, 2D 지도 비교가 아래**에 함께 표시 — 벤더 Module 4 스타일로 survey 전체에서 compensation 효과를 확인), "Batch"(여러 파일 일괄 compensation + 개선율 비교)
- **오른쪽 패널**: 구간 선택(Add/Remove/Clear segment, 추가된 구간 목록, "center segments" 체크박스), 모델 파라미터(method/grid resolution/coverage threshold/max variation/ trim outliers), Scan 버튼과 후보 리스트(주 헤딩 커버 여부 표시), Build/Save/Load Calibration, Compensate & Save
- **하단**: 실행 로그

7.2 사용 순서

1. "Load -pre.csv..."로 survey 파일을 연다 — Segment Selection 탭에 Mag 시계열과 비행 경로가 뜬다.

2. "Scan" 버튼으로 후보 구간을 찾거나, 플롯 위에서 직접 드래그해 구간을 고른다. 드래그하는 동안 오른쪽 비행 경로 플롯에서 그 구간이 측선상 어디에 해당하는지 바로 확인할 수 있다. 후보 리스트에서 항목을 클릭하면 선택 구간이 자동으로 반영된다.

3. 구간 하나로는 자기평탄도와 헤딩 다양성을 동시에 만족하기 어려울 때: 선택한 구간에서 "Add segment"를 눌러 목록에 담고, 2번 단계를 반복해 여러 구간을 계속 추가한다 ("Remove selected segment"/"Clear all segments"로 수정 가능). "center segments" 체크박스(기본 켜짐)는 서로 다른 지점에서 고른 구간의 실제 지자기장 레벨 차이를 보정해 합치기 전에 공통 기준선으로 맞춰준다 — 끄면 그 차이가 heading 효과로 오인돼 모델이 심하게 망가질 수 있으니(8.6절 참고) 다구간을 쓸 때는 켜둔 채로 두는 것을 권장한다.

4. "Build calibration"으로 캘리브레이션 모델을 만든다 — 목록에 추가된 구간이 있으면 그 구간들을 모두 이어붙여 하나의 모델로 만들고, 없으면 현재 드래그 선택 하나만 쓴다. 품질 리포트와 Calibration Quality 탭 플롯이 갱신된다. 필요하면 "trim outliers" 체크박스를 켠다.

5. 결과가 만족스러우면 "Save calibration..."으로 `.npz` 파일로 저장한다 (이후 같은 프로젝트의 다른 파일에는 "Load calibration..."으로 재사용).

6. "Compensate & save..."로 현재 로드된 survey 전체에 모델을 적용하고 `-comp.csv`를 저장한다 — GPS 좌표가 있으면 2D Comparison 탭에 전후 비교 지도가 뜬다.

7. "Batch" 탭에서 "Add files..." 또는 "Add folder..."로 현장에서 취득한 모든 파일을 불러온 뒤 "Compensate All"을 누르면, 파일별 motion-noise 개선율 표가 나온다. 개선율이 고르게 좋으면 캘리브레이션을 채택하고, 낮은 파일이 있으면 2~3단계로 돌아가 구간을 다시 골라본다.

8. 부록: Heading Error Compensation 이론

8.1 Heading 정의

MagArrow 고정 xyz 좌표계에서 지자기장 방향 B = (CompassX, CompassY, CompassZ)에 대해, θ = arccos(Bz / |B|) (polar), φ = atan2(By, Bx) (azimuth)로 정의한다. 이 각도쌍이 그 순간의 "헤딩"이다.

8.2 Heading Error

MagArrow는 total-field(스칼라) 자력계이므로 이상적으로는 방향과 무관하게 같은 값을 읽어야 하지만, nonlinear Zeeman shift 등 물리적 원인과 센서/플랫폼의 자성 재질로 인해 실제 판독값은 헤딩의 함수 ΔB(θ, φ)만큼 흔들린다. 이 프로그램의 목표는 ΔB(θ, φ)를 추정해서 매 샘플의 Mag 판독값에서 빼주는 것이다: `MagComp = Mag − ΔB(θ, ϕ)`.

8.3 캘리브레이션 모델: 왜 그리드 기반 평균인가

가장 단순한 접근은 캘리브레이션 구간의 (θ, φ, ΔB) 산점 데이터를 그대로 보간하는 것이다. 하지만 이렇게 하면 "캘리브레이션 데이터 자체에 모델을 적용했을 때 평평해야 한다"는 품질 검사가 항상 trivial하게 통과해버린다 — 각 점의 최근접 이웃이 자기 자신이라 거리가 항상 0이기 때문이다. 이 프로그램은 (θ, φ)를 3차원 단위구 좌표로 임베딩한 뒤 각도 그리드(`grid_resolution_deg`, 기본 5도)로 먼저 평균을 낸다. 그러면 개별 raw 샘플은 자기 자신이 속한 grid bin의 평균과 비교되므로, heading과 무관한 잡음(구배, 드론 간섭 등)이 실제로 잔차에 드러난다 — 벤더 매뉴얼이 설명하는 Module 3 품질검사와 동일한 동작을 하게 만든 핵심 설계 결정이다.

8.4 nearest vs spline

그리드로 평균낸 뒤, 각 grid bin 값을 보간하는 방법이 두 가지다:

- **nearest** — 질의 헤딩에서 가장 가까운 grid bin 값을 그대로 쓴다. 벤더 프로그램의 기본값이며 "안전한" 선택이다(급격한 외삽이 없다).
- **spline** — 3차원 단위구에 임베딩된 grid bin들에 thin-plate spline(RBF)을 적합한다. Zhang et al. 논문이 제안한 biharmonic spline 보간의 실용적 근사다. 캘리브레이션 커버리지 범위 밖(질의점이 가장 가까운 bin에서 `--coverage-threshold-deg` 이상 떨어진 경우)에서는 spline도 안전하게 nearest로 대체(fallback)한다 — 벤더 매뉴얼이 "외삽은 사실상 불가능하다"고 명시한 원칙을 따른 것이다.

HaeNam 샘플 데이터에서 실측한 결과 spline이 nearest보다 self-compensation p2p가 더 낮았다 (1.77 nT vs 2.38 nT) — 5장 참고.

8.5 캘리브레이션 자료: 양보다 질 — 이상치 트리밍의 근거

"캘리브레이션 구간을 통째로 쓰는 것과, 그중 변화(variation)가 큰 부분을 제거해서 max variation을 줄이는 것 중 어느 쪽이 나은가?"라는 질문에 대한 답이다. 결론부터 말하면 **자료 개수 자체는 목표가 아니다.** 벤더 매뉴얼 Module 3 절은 이 둘이 서로 상충한다고 직접 명시한다: "Selecting more data points can include more headings in the calibration file, however, is also more likely to contain non-heading-error-related field variations ... If spikes occur far away from headings of interest, the data should still be OK." 즉 벤더 스스로도 "관심 있는 헤딩에서 멀리 떨어진 스파이크라면 놔둬도 괜찮다"고 조건부로만 허용한다 — 다르게 말하면, **관심 헤딩(특히 survey의 주 비행 방향)에서 벗어난 스파이크만 안전하게 무시할 수 있고, 그 방향의 유일한 관측치라면 지워선 안 된다는 뜻이다.**

이 프로그램의 `--trim-outliers`는 이 원칙을 정량적으로 구현한다: 각 각도 그리드 칸(bin) 안에서만 이상치를 판단하고(median absolute deviation 기준), 표본이 충분한(기본 3개 이상) 칸에서만, 그것도 칸의 절반을 넘지 않는 선에서만 제거한다. 표본이 1~2개뿐인 칸(= 자주 담기지 않는 희귀한 헤딩)은 아무리 극단적인 값이라도 절대 건드리지 않는다 — 그 칸을 비우면 바로 그 헤딩에서 compensation이 안 되는

구멍이 생기기 때문이다. 결과적으로 "많이 반복된 헤딩에서는 잡음만 걸러내고, 희귀한 헤딩은 있는 그대로 보존"하는 전략이 된다.

실전 함의: 좋은 캘리브레이션 구간을 만드는 두 가지 손잡이는 서로 다른 문제를 푼다. **구간을 넓히는 것**은 커버리지(특히 주 비행 방향 포함 여부)를 늘리기 위한 것이고, **이상치 트리밍**은 그렇게 넓힌 구간 안에 섞여 들어온 heading과 무관한 잡음(공간적 자기장 구배, 드론 근접 등)을 제거하기 위한 것이다. 구간을 넓히기만 하고 트리밍을 안 하면 잡음이 누적되고, 트리밍만 하고 구간을 넓히지 않으면 애초에 커버할 헤딩이 부족하다 — 둘을 함께 쓰는 것이 실전에서 가장 효과적이다.

8.6 여러 구간을 이어붙인 캘리브레이션 — 다구간 조합과 기준선 보정

한 구간 안에서 "자기평탄도"와 "헤딩 다양성"을 동시에 만족시키기는 실제 survey 자료에서 생각보다 어렵다. 지난 홍천 프로젝트에서 실제로 잘 작동했던 제조사 프로그램의 `-cal.csv` 파일을 역추적해보니, 하나의 긴 연속 구간이 아니라 **짧고 서로 떨어진 여러 구간을 이어붙인 것**이었다(행 순서가 원본 파일 안에서 앞뒤로 여러 번 오간다 — 34번의 방향 반전 확인). `calibrate`의 `--start/--end`에 여러 값을 짝지어 주면 이 방식을 그대로 재현할 수 있다: 짧은 구간 여러 개가 각각 평탄하기만 하면 되고, 헤딩은 구간들에 걸쳐 합산된다.

다만 서로 다른 지점에서 고른 구간은 실제 지자기장 레벨도 다른 경우가 흔하다. 홍천 자료로 직접 검증한 예: 같은 비행 파일 안에서 겨우 5분 이내로 떨어진 4개 구간(각각 개별적으로는 self-compensation p2p 0.46~0.58 nT로 매우 우수)의 Mag 중앙값이 서로 최대 ~12 nT나 차이가 났다. 캘리브레이션 모델은 풀링된 전체 자료에서 **하나의** 전역 기준선(median)만 계산하므로, 이 정도의 공간적 차이를 그대로 heading 효과로 오인해버려 모델이 심하게 망가진다 — 실측 결과, 기준선 보정 없이 이 4개 구간을 그대로 합쳤을 때 self-compensation p2p가 17.8 nT까지 치솟았고 (개별 구간의 30배 이상), 실제 compensation을 적용해도 선회 구간 잡음이 오히려 34.8% 악화됐다. 이는 벤더 매뉴얼이 "멀리 떨어진 두 지역의 자료를 합칠 때는 오프셋 보정이 필요할 가능성이 높다"고 명시한 경고와 정확히 같은 현상이며, 반드시 서로 다른 비행(flight)일 때만이 아니라 **같은 파일 안에서도** 구간이 충분히 떨어져 있으면 똑같이 발생한다는 뜻이다.

이 프로그램은 여러 구간을 조합할 때 기본적으로 **각 구간 자체의 median을 먼저 빼서 공통 기준선에 맞춘 뒤** 풀링한다 — 벤더 매뉴얼이 권고하는 바로 그 오프셋 보정을 자동으로 수행하는 것이다. 위와 동일한 4개 구간에 이 보정을 적용하면 self-compensation p2p가 0.56 nT로 정상화되고, 선회 구간 잡음도 오히려 0.4% 개선된다. 구간이 하나뿐일 때는 이 보정이 수학적으로 아무 효과가 없으므로(자기 자신의 median을 빼고 다시 더하는 셈) 단일 구간 워크플로에는 영향이 없다. 필요 시 `--no-center-segments`로 끌 수 있으나, 여러 구간을 합칠 때는 끄지 않는 것을 권장한다.

8.7 auto-calibrate — 구간 탐색·조합·검증·저장을 한 번에

8.1~8.6절에서 설명한 워크플로 전체(구간 후보 스캔 → 품질 확인 → 안 되면 다른 구간/여러 구간 조합으로 재시도 → 통과하면 저장)를 자동화한 것이다. 벤더 GUI의 "Calibration Data Quality Check" 화면도 본질적으로 같은 루프다 — 사용자가 구간을 고르고, Pass/Fail을 보고, 안 되면 다른 구간으로 New Params를 눌러 반복하고, Pass가 뜨면 Select를 누른다. `auto-calibrate`는 이 루프를 이 프로그램이 이미 계산하는 정량 지표(self-compensation p2p, survey 주 헤딩 커버 여부, 헤딩 미달 비율)로 자동 반복하고, **실제로 통과했을 때만** 파일로 저장한다(`--force-save` 없이는 미달 상태를 저장하지 않는다) — 8.2절에서 강조한 원칙 그대로, 지표만 보고 성공을 선언하지 않는다.

`--window-size`는 넓은 값부터 순서대로 시도하다 실패하면 자동으로 더 좁은 값으로 넘어간다 (기본값 3000, 1500, 750, 300). 캘리브레이션 구간이 얼마나 넓어도 "깨끗함"을 유지할 수 있는지는 현장의 실제 자기장 공간적 구배에 달린 문제라 한 값으로 못 박을 수 없다 — 턴 구간의 헤딩 스윙 전체를 담으려면 넓은 구간(수천 샘플)이 유리하다는 게 일반적인 직관이지만, 실측해보면 정반대일 수도 있다: 홍천 자료로 검증한 결과 3000/1500샘플 구간은 모조리 실패했다(p2p 최대 68 nT) — 이 survey 지역의 실제 자기장 구배가 턴 하나의 비행 거리보다도 가파르기 때문이다. 750샘플로 자동 전환되고 나서야 서로 다른 두 지점(약

6000샘플, 10분 이상 떨어짐)의 구간 2개를 조합해 self-compensation p2p 1.4 nT, 헤딩 미달 비율 5.1%로 통과했다. turn-only 검증(6.3절 참고, 실제 헤딩 변화가 큰 구간만 골라 노이즈 개선율을 따로 계산)으로 재확인한 실제 개선율은 +7.9% — 정적 지표뿐 아니라 실제 compensation 효과로도 확인됐다.

구간 조합 알고리즘: 각 window-size에서 scan이 랭킹한 후보 중 상위 최대 60개(`--candidate-pool-size`)를 훑으며, 이미 채택한 구간과 30% 넘게 겹치는 후보는 건너뛰고(같은 구간의 미세하게 다른 슬라이스는 새 헤딩 커버리지를 더해주지 않는다), 나머지는 8.6절의 기준선 보정을 적용해 채택 후보 목록에 추가해보고, 그 결과 합쳐진 전체의 self-compensation p2p가 --max-variation을 넘으면 그 후보만 취소하고 다음 후보로 넘어간다. 합친 결과가 p2p 통과 + survey 주 헤딩 커버 + 헤딩 미달 비율이 --coverage-target 이하가 되면(기본 10%) 그 자리에서 멈춘다(최대 --max-candidates개, 기본 15개까지 채택). **주의:** scan 후보 상위권이 survey의 한 지점에 쏠려 있는 경우가 흔해서(가장 깨끗한 한 구역의 슬라이스가 온통 상위를 차지) 후보 풀을 채택 개수보다 훨씬 크게 잡아야 실제로 다른 지점의 구간까지 도달한다 — 홍천 자료에서 풀 크기를 15개로 좁혔을 때는 헤딩 미달 비율이 50%에서 멈춰 있었지만, 60개로 넓히자 비로소 5.1%까지 내려갔다.

사용법: `python -m mag_heading auto-calibrate --pre <survey>-pre.csv --out project.cal.npz --max-variation 2.0`. GUI에서는 Segment Selection 탭 오른쪽의 "Auto-build" 패널에서 window sizes/coverage target/max segments를 조정하고 "Auto-build calibration" 버튼을 누르면 된다 — 통과하면 채택된 구간이 목록에 채워지고 저장 대화상자가 자동으로 뜨고(파일명은 직접 확인 후 지정), 통과하지 못하면 경고 창과 함께 최선의 시도 결과만 화면에 표시되고 저장은 하지 않는다.

8.8 level-survey — 측선간 헤딩/방향 오차 레벨링

8.1~8.7절의 compensation은 **한 파일 안에서** 그 순간의 자세(theta, phi)로부터 ΔB를 계산해 빼는 방식이다. 이 보정이 얼마나 잘 듣는지는, 그 특정 자세 부근에서 캘리브레이션 비행이 얼마나 촘촘하게 샘플을 남겼는지에 달려 있다. 실제 HaeNam M350 자료에서는, 잘 검증된 ΔB(theta, phi) 캘리브레이션(turn-only 잡음 +7.3% 개선, 42/42 파일 양성)을 적용한 뒤에도 survey 전체에서 **북행/남행 측선 사이에 약 5.6 nT의 일정한 오프셋**이 남았다 — 이 플랫폼의 실제 반대 방향 자세(theta 남행 ~63도, 북행 ~19도)가 캘리브레이션 비행(수평 loiter)이 다룬 자세 범위보다 훨씬 벌어져 있었기 때문이다. 캘리브레이션을 여러 방식으로 다시 만들어봐도(더 촘촘한 커버리지, survey 실제 커버리지에 맞춘 구간, loiter 대신 survey 자체 턴 구간 사용) 이 잔차는 사라지지 않고 위치만 옮겨 다녔다 — 캘리브레이션 자체의 한계가 아니라, 그 두 지점에서 fit된 곡면의 불확실성이 한계였기 때문이다.

이 문제는 더 나은 캘리브레이션 비행으로 풀리지 않는다. survey 자신의 반대 방향 측선들을 직접 비교하면 이 오프셋을 훨씬 더 튼튼하게(캘리브레이션 비행 한 번이 아니라 수십 개의 독립적인 측선 쌍을 평균해서) 잴 수 있다. `level-survey`는 이 아이디어를 구현한다: 여러 파일을 GPS 좌표 하나의 공통 기준으로 합쳐 측선을 검출하고, 겹치는 국소 구역마다 value = a + b·dx + c·dy + offset·방향으로 평면을 적합해 실제 지형 경사(평면 항)와 방향에 따른 계통 오차(offset 항)를 분리한 뒤, 잔차가 가장 작은(가장 "조용한") 구역들의 offset을 강건 중앙값으로 취합한다. 별도 자력탐사 자료처리 앱의 `processing/leveling.py` "local_plane" 방식(실측 HaeNam M350 블록에서 stripe ratio 93% 감소)을 그대로 이식한 것이다.

구역별 offset의 산포가 offset 값 자체보다 크면 이 보정은 자동으로 "신뢰할 수 없음" 경고를 내고 기본적으로 아무 것도 쓰지 않는다 — 그 경우 남은 값은 방향에 따른 계통 오차가 아니라 측선별 개별 레벨 잡음일 가능성이 높기 때문이다(8.2절의 원칙과 동일: 지표만 보고 성공을 선언하지 않는다). `--force`로 강제 적용할 수 있으나, 출력된 경고와 offset/산포 수치를 먼저 확인해야 한다.

사용법: `python -m mag_heading level-survey --pre-glob "<folder>/*-pre.csv" --cal project.cal.npz --out-dir <out>` (캘리브레이션까지 한 번에) 또는 이미 compensate를 끝낸 `-comp.csv` 묶음에 `--cal` 없이 바로 실행(레벨링만 추가). 적용되면 각 파일에 `MagLeveled` 컬럼이 새로 추가되어 저장된다 — 기존 `Mag`/`MagComp`는 그대로 남는다.

9. 문제 해결

옵션	설명
missing required columns [...]	-pre.csv에 Mag/CompassX/CompassY/CompassZ 중 일부가 없다. 벤더 프로그램 Module 1을 먼저 거쳤는지 확인한다.
compass vector magnitude is zero	CompassX=CompassY=CompassZ=0인 샘플이 있다. 원본 취득 데이터 오류일 가능성이 높다 — 해당 구간을 캘리브레이션/compensation 범위에서 제외한다.
window_size exceeds data length	scan의 --window-size가 -pre.csv 전체 샘플 수보다 크다. 더 작은 값으로 다시 시도한다.
self_compensation_pass: False가 계속 나옴	--max-variation을 너무 엄격하게 잡았거나, 이 survey 지역 자체의 자기장 구배가 커서 짧은 구간으로도 완전히 평평해지기 어려운 경우다. 여러 후보의 상대적 순위로 판단하고, 목표로 하는 이상체 신호 크기 대비 잔차가 충분히 작은지로 실용적으로 판단한다.
GUI가 뜨지 않음 / Tkinter 오류	Python이 Tkinter 없이 빌드된 배포판(일부 minimal 배포판)일 수 있다. 표준 python.org 설치본을 사용한다.
batch-compensate에서 일부 파일만 개선율이 낮음	그 파일들이 캘리브레이션 구간에서 충분히 다루지 못한 헤딩(예: 다른 방향의 측선)을 포함하고 있을 가능성이 높다. 해당 파일도 포함해서 `scan`을 survey_df로 다시 돌리거나, 그 파일 자체의 헤딩 분포를 `plot-heading`으로 확인한다.
no *-pre.csv files found in (GUI)	"Add folder..."는 폴더 안에서 파일명이 정확히 `*-pre.csv` 패턴인 파일만 찾는다. 파일명이 다르면 "Add files..."로 직접 선택한다.

참고문헌

- Geometrics, Inc., Survey Data Processing Program User Guide.
- Zhang, R., Hanselmann, F., Fochs, R., Lawrence, M., Hurley, K., Prouty, M. (2022). "Motion noise reduction in drone magnetometry using a dead-zone-free total-field magnetometer system." The Leading Edge, 41(7), 306–310.